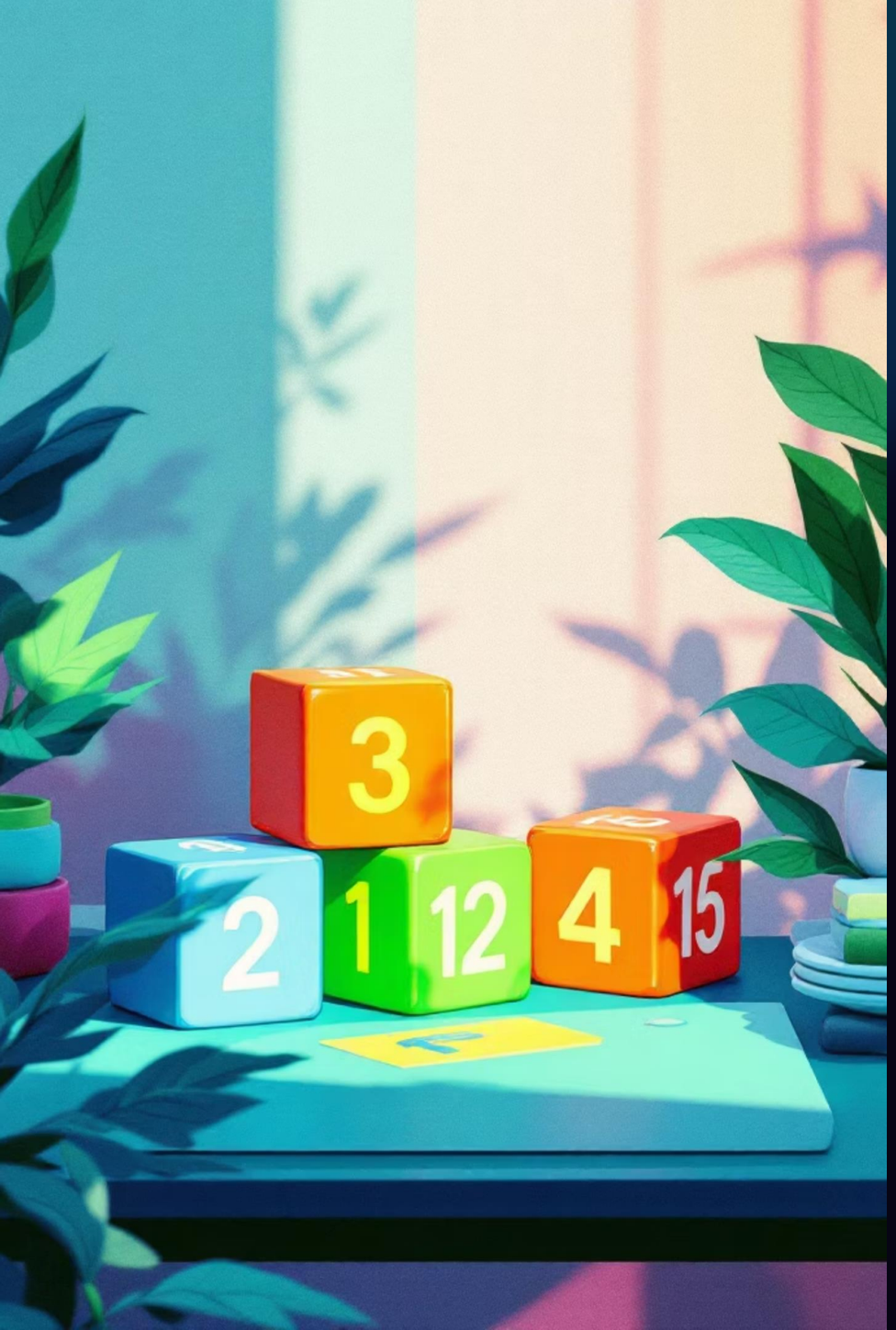
Day 6 – DSA from Zero

Prime Numbers

An easy, beginner-friendly explanation of prime numbers — what they are, how to check them, and how to do it efficiently. Welcome to MK TechVerse 🚀

🚀 DAY 6 | EASY CONCEPT | JAVA CODE

Made with GAMMA



What Is a Prime Number?

A **prime number** is a whole number greater than 1 that can only be divided evenly by **1** and **itself**. No other number divides it without leaving a remainder. Think of it as a number with "no factors" other than the obvious ones.

✅ Prime Numbers

2, 3, 5, 7, 11, 13...

Only divisible by 1 and themselves.

❌ Not Prime (Composite)

4, 6, 8, 9, 10, 12...

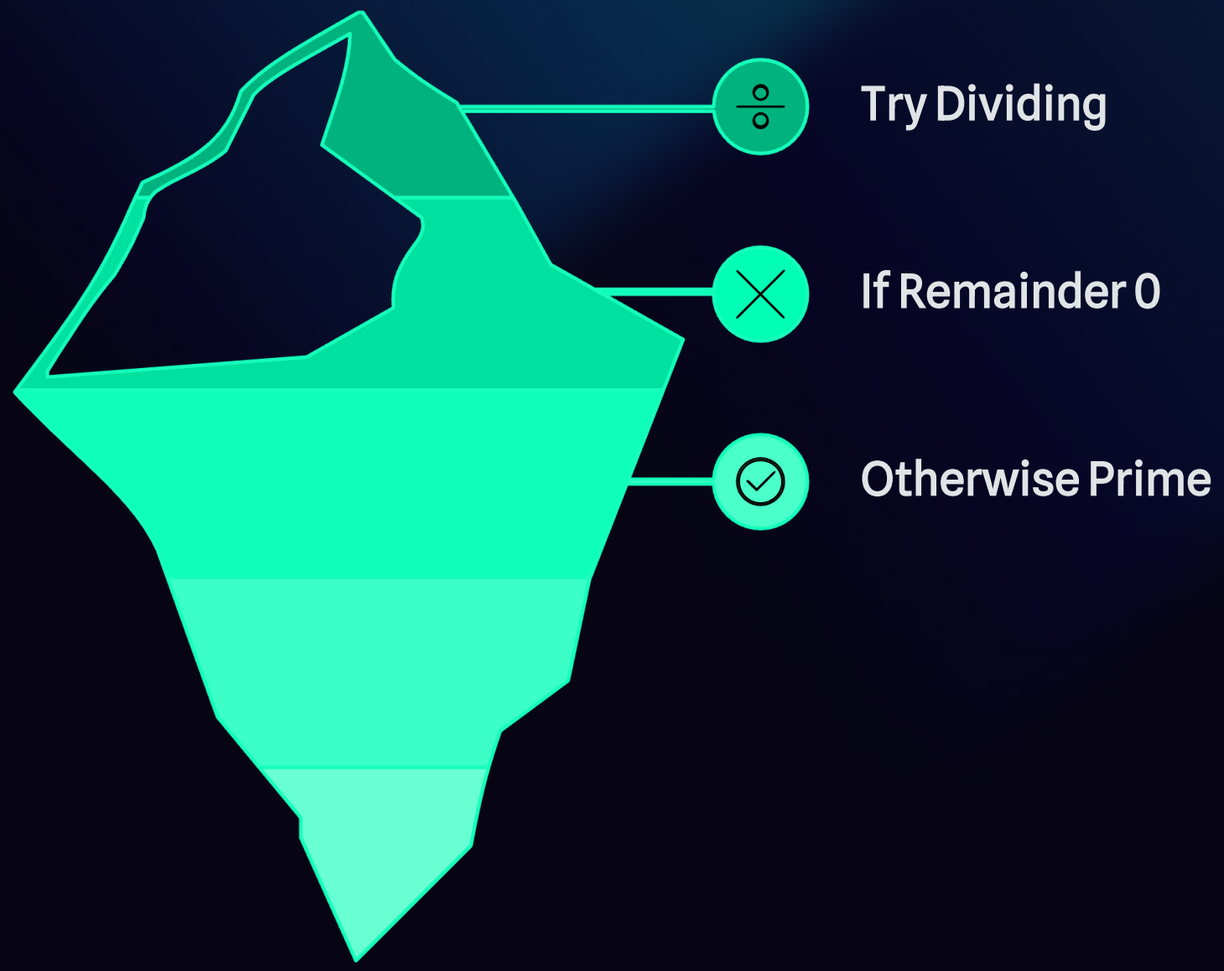
Have other divisors besides 1 and themselves.

Special case: The number **1** is *not* prime. The smallest prime number is **2** — and it's the only even prime!

Made with GAMMA

The Simple Idea Behind Checking Primes

Before writing any code, let's build the intuition. The core idea is beautifully simple — try to divide the number by smaller numbers and see what happens.



This trial-division approach is the foundation of every prime-checking algorithm. The key question is: **how far do we need to check?** That's where optimization comes in — and it makes a huge difference in performance.

Made with GAMMA

Let's Walk Through Examples

Before diving into code, let's manually trace through two numbers to build our intuition. This is exactly the kind of thinking interviewers want to see.

Checking 7 → Prime ✅

Try dividing 7 by numbers starting at 2:

- 7 ÷ 2 = remainder **1** ❌
- 7 ÷ 3 = remainder **1** ❌

Nothing divides evenly → **7 is Prime!**

Checking 8 → Not Prime ❌

Try dividing 8 by numbers starting at 2:

- 8 ÷ 2 = remainder **0** ✅ Divides!

We found a divisor → **8 is Not Prime!** No need to check further.

Made with GAMMA

Basic Approach — The Slow Way

The most straightforward method is to check **every number** from 2 up to **n – 1**. If any of them divide `n` evenly, it's not prime.

How It Works

Loop from `i = 2` to `i = n-1`. Check if `n % i == 0`. If yes → Not Prime. If the loop finishes with no divisor → Prime.

Time Complexity: O(n)

For `n = 1,000,000`, this does up to **999,999 checks**. That's a lot of work — and it gets worse as `n` grows.

Why It's Not Efficient

We're doing far more work than needed. Many of those checks are redundant. We can do **much better** with a simple mathematical insight.

Made with GAMMA

Smart Approach — Check Only Up to √n

Here's the key insight: **factors always come in pairs.** If `a × b = n`, then at least one of them must be ≤ √n. So we never need to check beyond √n!

Loop Condition

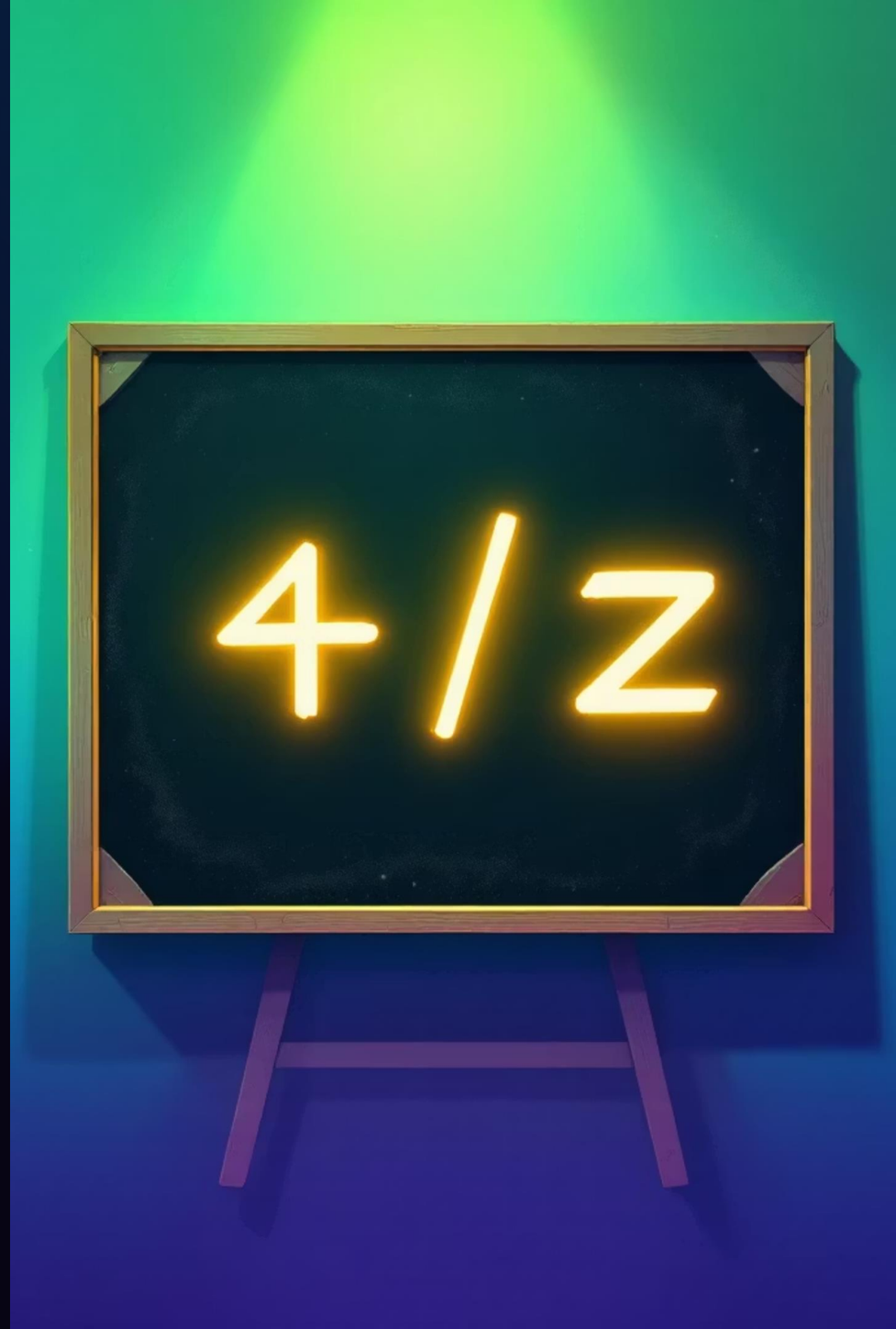
Instead of `i < n`, use `i * i <= n` — this checks up to √n without needing a square root function.

Massive Speedup

For `n = 1,000,000`, we go from **999,999 checks** down to just **1,000 checks**. That's a 1000× improvement!

Time Complexity

O(√n) — dramatically faster than O(n). This is the standard approach used in interviews and competitive programming.



Why √n Works — The Factor Pair Insight

Let's use **36** as our example. Every factor has a "partner" — when you multiply them, you get 36. Notice what happens at $\sqrt{36} = 6$:

Factor Pair	Observation
1 × 36	1 is below $\sqrt{36}$
2 × 18	2 is below $\sqrt{36}$
3 × 12	3 is below $\sqrt{36}$
4 × 9	4 is below $\sqrt{36}$
6 × 6	Both equal $\sqrt{36}$ ← Meeting point!
9 × 4	Repeats — we've seen this!

The pattern: After $\sqrt{n}$, factor pairs just repeat in reverse order. So checking beyond $\sqrt{n}$ is completely unnecessary — we've already found (or ruled out) all possible factors.

Made with GAMMA

The Code — Java Implementation

Here's the complete, clean Java code using the √n optimization. Notice how `i * i <= n` replaces the need for `Math.sqrt()`.

```
int n = 7;

// Edge case: 1 and below are NOT prime
if (n <= 1) {
    System.out.println("Not Prime");
    return;
}

boolean isPrime = true;

// Check from 2 up to √n (i*i <= n)
for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) {
        isPrime = false;
        break; // Found a divisor, stop early
    }
}

if (isPrime) {
    System.out.println("Prime");
} else {
    System.out.println("Not Prime");
}
```

Key Line

`i * i <= n` — this is the √n optimization. It stops the loop at the square root without calling `Math.sqrt()`.

Why `break`?

Once we find *any* divisor, we know the number is not prime. No need to keep checking — exit early to save time.

Made with GAMMA

Time & Space Complexity

Understanding complexity is essential for DSA interviews. Here's the full breakdown of our optimized prime-checking solution.

$O(\sqrt{n})$

Time Complexity

We iterate at most $\sqrt{n}$ times. For $n = 10^6$, that's only 1,000 iterations instead of 1,000,000.

$O(1)$

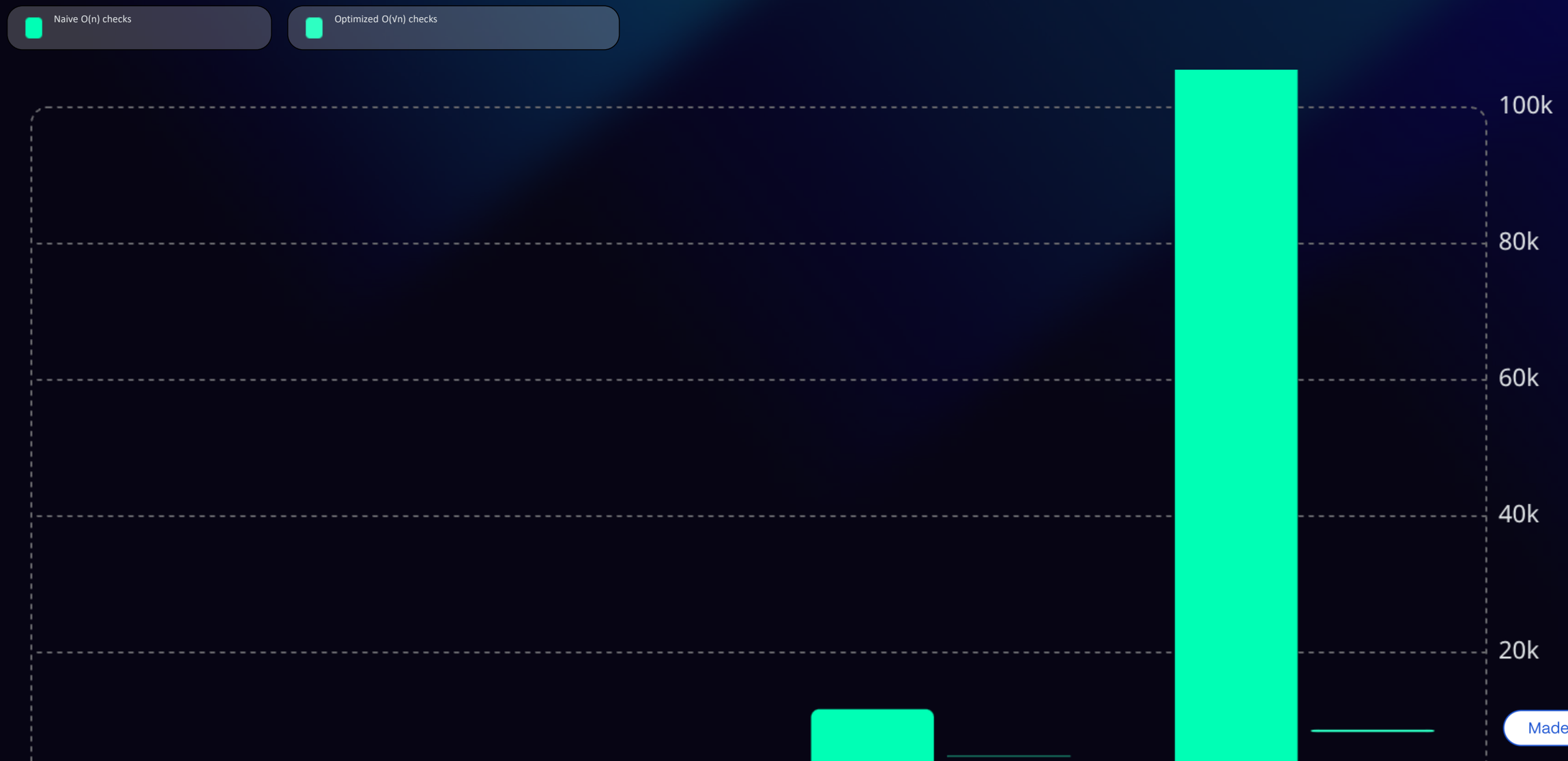
Space Complexity

We only use a single boolean variable `isPrime`. No extra data structures — constant space.

1000×

Speedup vs Naive

Compared to the O(n) approach, the $\sqrt{n}$ optimization gives a massive performance gain for large inputs.



Made with GAMMA

🏆 Mini Challenge — Try It Yourself!

Before moving on, test your understanding. Trace through the algorithm by hand for these two numbers. Can you determine if they're prime?

1

Check n = 11

$\sqrt{11} \approx 3.3$, so check i = 2 and i = 3.

- 11 ÷ 2 → remainder?
- 11 ÷ 3 → remainder?

Answer: 11 is Prime ✅ — no divisors found!

2

Check n = 12

$\sqrt{12} \approx 3.4$, so check i = 2 and i = 3.

- 12 ÷ 2 → remainder?

Answer: 12 is Not Prime ❌ — divisible by 2!

Homework: Modify the code to print all prime numbers from 1 to 100. This is called the **Sieve of Eratosthenes** concept — we'll cover it soon! 🚀

Made with GAMMA